

Linked List in C++

Data Structures · CT077-2-3-DSTR · Full Lesson Notes

Singly Linked

Doubly Linked

Circular Linked



1. Array Review & Limitations

WHAT IS AN ARRAY?

A data structure that stores items **in a row** (contiguous memory). Items are accessed using an **index** (position number starting at `[0]`).

5 [0]

3 [1]

7 [2]

2 [3]

9 [4]

ARRAY DISADVANTAGES (4 KEY PROBLEMS)

- 1 **Single data type only** — all elements must be the same type (e.g. all `int`)
- 2 **Fixed size** — declared as `array[5]` ; cannot grow or shrink at runtime
- 3 **Difficult to insert / delete** — all subsequent elements must shift to maintain order
- 4 **Requires an index to access** — cannot search without knowing the position first



2. What is a Linked List?

DEFINITION

A **linked list** is a data structure consisting of **nodes**. Each node holds two parts:

- **data** — the actual value stored
- **pointer** — the memory address of the *next* node

The list always starts with a **Head** pointer. The last node points to `nullptr`.



Analogy: Like a train — a driver's coach (Head) connects to passenger coaches (nodes). You can add or remove coaches without affecting the rest.

WHY USE LINKED LIST OVER ARRAY?

- ✓ Dynamic size — grows and shrinks at runtime
- ✓ Efficient insert/delete — just re-link pointers
- ✓ No wasted memory allocation

REAL-LIFE EXAMPLES

- **Contact List** — each contact is a node; `next` points to the next contact
- **To-Do List** — tasks chained together, easy to add/remove
- **Music Player (Prev/Next)** — each song node links to the next (Taylor Swift → Doja Cat → Adele → `nullptr`)



3. Creating a Linked List (2 Steps)

STEP 1 — DEFINE THE NODE STRUCTURE

Each node is a `struct` (or `class`) containing data fields and a pointer to the next node.

```
struct Node {  
    int data;  
    Node* next;  
};
```

The `next` member stores the address of the next `Node` in the list.

Contact Book example — node with multiple data fields:

```
struct Contact {  
    string name;  
    string phoneNumber;  
    Contact* next;  
};
```

STEP 2 — CREATE AND INITIALISE THE HEAD POINTER

Declare a `head` pointer and set it to `nullptr` to represent an empty list.

```
Node* head = nullptr;
```

 **Rule:** A linked list MUST always start with a head pointer. Without it, you cannot access any node.

4. Linked List Operations

① Insertion at the Beginning

Insert a new node *before* the current head. Time: $O(1)$

1. Create a new node
2. Set `newNode->next = head`
3. Update `head = newNode`

```
void insertAtBeginning(Node*& head, int val) {  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    head = newNode;  
}
```

② Insertion at the End

Traverse to the last node (`next == nullptr`) and append. Time: $O(n)$

1. Create a new node with `next = nullptr`
2. Traverse to the last node
3. Set `lastNode->next = newNode`

```
void insertAtEnd(Node*& head, int val) {  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = nullptr;  
    if (!head) { head = newNode; return; }  
    Node* curr = head;  
    while (curr->next) curr = curr->next;  
    curr->next = newNode;  
}
```

③ Insertion at a Specific Position

Insert between two existing nodes by re-routing pointers.

1. Traverse to the node *before* the target position
2. Set `newNode->next = prevNode->next`
3. Set `prevNode->next = newNode`

 **Order matters** — always link the new node forward first, then re-link the previous node. Doing it backwards loses the rest of the list.

④ Deletion of a Node

Remove a node and reconnect the surrounding nodes. Always free memory with `delete` .

1. Traverse to the node *before* the target
2. Set `prevNode->next = targetNode->next`
3. `delete targetNode` to free memory

```
void deleteNode(Node*& head, int val) {
    if (!head) return;
    if (head->data == val) {
        Node* tmp = head;
        head = head->next;
        delete tmp; return;
    }
    Node* curr = head;
    while (curr->next && curr->next->data != val)
        curr = curr->next;
    if (curr->next) {
        Node* tmp = curr->next;
        curr->next = tmp->next;
        delete tmp;
    }
}
```

⑤ Searching for a Value

Use a **current** pointer to walk through each node. Time: O(n)

1. Set `current = head`
2. While `current != nullptr`: check `current->data`
3. If match → found! If not → `current = current->next`

```
bool search(Node* head, int val) {
    Node* curr = head;
    while (curr) {
        if (curr->data == val) return true;
        curr = curr->next;
    }
    return false;
}
```

⑥ Reversing the Linked List

Flip all pointers so the last node becomes the new head. Uses 3 pointers.

Use: `prev = nullptr` , `curr = head` , `next = nullptr`

While traversing: save `next` , flip `curr->next = prev` , advance both

At the end, set `head = prev`

```
void reverse(Node*& head) {  
    Node* prev = nullptr;  
    Node* curr = head;  
    while (curr) {  
        Node* next = curr->next;  
        curr->next = prev;  
        prev = curr;  
        curr = next;  
    }  
    head = prev;  
}
```



5. Using the STL <list> in C++

C++ provides a built-in doubly linked list via `#include <list>`. Common methods:

Method	Action
<code>push_front(val)</code>	Insert at beginning
<code>push_back(val)</code>	Insert at end
<code>remove(val)</code>	Delete node with value
<code>reverse()</code>	Reverse the list
<code>size()</code>	Return number of nodes

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<string> artists;

    artists.push_front("Taylor Swift"); // Insert at start
    artists.push_back("Adele");          // Insert at end
    artists.push_back("Doja Cats");      // Insert at end

    artists.reverse();                   // Reverse the list
    artists.remove("Adele");             // Delete by value

    for (const auto& a : artists)
        cout << a << " -> ";
    cout << "nullptr";
}
```



6. Quick Reference Summary

Node structure

data + `next*` pointer. Defined as a `struct` .

Head pointer

Entry point. Initialised to `nullptr` for empty list.

Insert beginning

$O(1)$ — point new node to old head, update head.

Insert end

$O(n)$ — traverse to last node, then append.

Delete node

Bypass node by re-linking prev. Always `delete` freed node.

Traversal / search

$O(n)$ — walk with a `current` pointer until `nullptr` .

 **Key insight:** Linked lists excel at *dynamic* memory use and cheap insertions/deletions. Arrays excel at *random access* by index. Choose based on your use case.

Types of linked lists (upcoming lectures):

Singly linked

Doubly linked

Circular singly linked